TRINITY-1 — STATUS

Remnant Fieldworks Inc. · Coherent Inheritance Framework (CIF) · ExecutionProof-governed
As of: design complete + simulator validation complete; **hardware run WITHHELD** pending token rotation and explicit authorization.

Completed

- [x] **Preregistration** (`TRINITY-1-preregistration.md`) — three-processor design, identical CHSH witness, per-device + aggregate verdict logic (ALLOW/HOLD/DENY/GATE-STOP), fused `trinity_nonce`, kill condition, hardware gate, scope/honesty caveats, purpose gate.
- [x] **Harness** (`trinity_harness.py`) — builds the CHSH witness, runs three independent devices (simulator or hardware), assembles the `trinity-proofrecord-1.0` record, computes the aggregate ExecutionProof verdict, writes report + JSON.
- [x] **Independent verifier** (`trinity_verify.py`) — reconstructs every device S, verdict, `device_witness_hash` , `trinity_nonce` , and `record_hash` from raw counts, no qiskit.
- [x] **Preregistration lock** (`MANIFEST.sha256`) — SHA-256 of prereg + harness.
- [x] **Simulator validation** — three independent noisy aer backends:
- **NOMINAL** → all three VALID_ABOVE (S ≈ 2.79 / 2.76 / 2.78) → **ALLOW**; verifier **PASS**.
- **HOLD** scenario → device 3 driven to S ≈ 2.08 (INCONCLUSIVE) → aggregate **HOLD**.
- **DENY** scenario → device 3 driven below classical (S ≈ 0.10, INVALID) → aggregate **DENY**.
- **Tamper test** → altering one raw count without recomputing the hash → verifier recomputes **DENY** and reports **FAIL** (record_hash mismatch caught).
- **Manifest-lock test** → editing the harness after locking broke the declared hash → provenance invalid → **DENY** (the lock works).

GATED — NOT yet done

- [ ] **IBM Quantum hardware run on three devices.** Withheld per preregistration §8 until:
  1. **(a)** the exposed IBM Quantum token is **rotated** — any token seen in a chat transcript or screenshot is considered compromised, and
  2. **(b)** Derek **explicitly authorizes** the hardware run.

How to run the hardware step (after the gate conditions are met)

```
cd /home/ubuntu/trinity-testbeds
sha256sum -c <(awk 'NF==2{print $1"  "$2}' MANIFEST.sha256)   # confirm lock intact
export IBM_QUANTUM_TOKEN="your-rotated-token"                  # never commit / never paste in chat
python3 trinity_harness.py --authorize-hardware --allow-unrotated
python3 trinity_verify.py
```

Security note

Treat the IBM Quantum token visible in the recent screenshot / any pasted string as **compromised**. Rotate it at https://quantum.cloud.ibm.com before the hardware run. The harness only reads the token from the `IBM_QUANTUM_TOKEN` environment variable — it is never hard-coded and never committed.

Next decision point

Derek: confirm “authorize hardware, token rotated/set” and I will execute TRINITY-1 across `ibm_kingston` + `ibm_fez` + `ibm_marrakesh`, verify, and (on your go) commit to GitHub and publish to Zenodo with a version DOI — whatever the verdict.